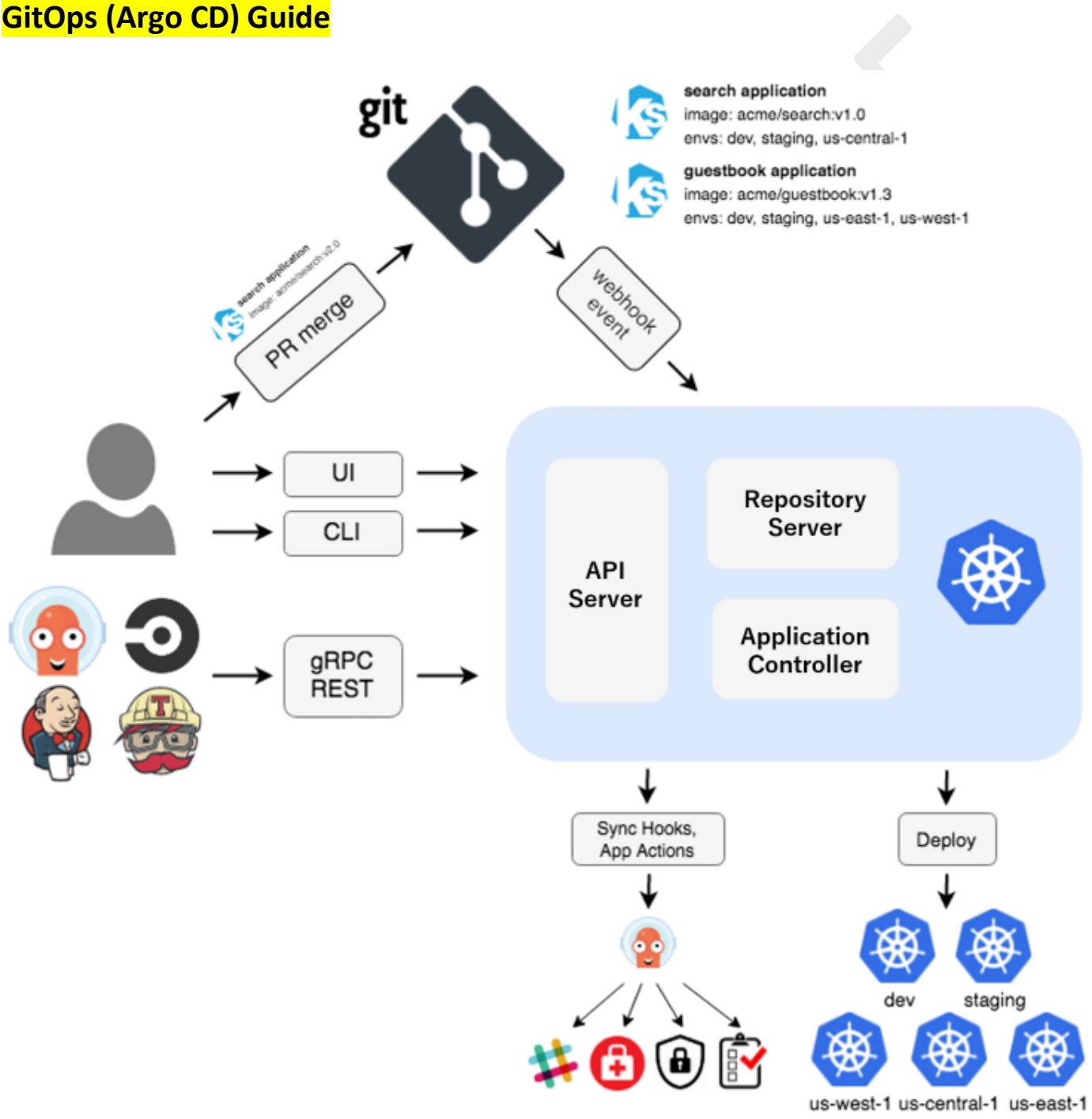
GitOps (Argo CD) Guide



What is Argo CD?

Argo CD (short for **Argo Continuous Delivery**) is a popular **declarative, GitOps-based continuous delivery tool** for Kubernetes. It helps automate the deployment and lifecycle management of applications on Kubernetes clusters by syncing the desired state defined in Git repositories with the actual state in the cluster.

Key Features of Argo CD:

- **GitOps approach**: The source of truth for your Kubernetes application manifests (YAML/Helm/Jsonnet) is stored in a Git repository. Argo CD continuously monitors this repo and automatically applies changes to your Kubernetes cluster, ensuring your cluster state matches what's in Git.
- **Declarative setup**: You declare your desired Kubernetes application manifests in Git, and Argo CD ensures the cluster reflects this.
- **Application syncing & health monitoring**: Argo CD provides real-time monitoring of the application state, highlighting differences between Git and cluster states, and supports automated or manual syncing.
- **Multi-cluster support**: Manage deployments to multiple Kubernetes clusters from a single Argo CD instance.
- **Support for various config formats**: Works with Helm charts, Kustomize, plain YAML, Jsonnet, and more.
- **Role-Based Access Control (RBAC)**: Secure access to applications and clusters.
- **Web UI & CLI**: Provides a user-friendly dashboard for visualizing application states and performing operations, plus CLI and API access.

How Argo CD Works (Overview):

1. **Define your app manifests in Git** — your Kubernetes YAML/Helm charts etc.
2. **Register your Kubernetes clusters with Argo CD.**
3. **Create an Argo CD Application** linking Git repo + target cluster + namespace.
4. Argo CD **monitors Git repo for changes**.
5. When changes happen, Argo CD **syncs the cluster state** with the Git-defined desired state.
6. You can view the app health, sync status, and history in the UI or CLI.

Why use Argo CD?

- Automates deployment, reducing manual errors.
- Keeps infrastructure and application configurations version-controlled and auditable.
- Enables progressive delivery strategies (canary, blue-green) with other tools like Argo Rollouts.
- Provides a clear audit trail of changes via Git commits.
- Simplifies multi-environment management (dev/stage/prod).

In short: Argo CD is a Kubernetes-native GitOps continuous delivery tool that makes your Kubernetes deployments automated, consistent, and easy to manage from Git repos.

There been a lot of CI/CD systems before ArgoCD (Jenkins, Teamcity, CircleCI, etc.) What added value ArgoCD brought?

Jenkins, TeamCity, CircleCI, and many others have been around for years, providing CI/CD functionality. So, **what unique value does Argo CD bring to the table?**

Here's what sets Argo CD apart, especially in the Kubernetes ecosystem:

1. Native Kubernetes GitOps Model

- Traditional CI/CD tools like Jenkins or CircleCI typically focus on *building, testing,* and *pushing* artifacts (e.g., Docker images), sometimes deploying via imperative scripts or pipelines.
- Argo CD **focuses exclusively on continuous delivery** using **GitOps**, a paradigm where:
  - Git is the *single source of truth* for the **desired state** of your Kubernetes cluster.
  - Argo CD continuously monitors Git and **automatically syncs** your cluster to match the repo.
- This **declarative approach** aligns perfectly with Kubernetes' own declarative nature and solves deployment drift issues better than imperative commands.

2. Declarative, Automated Sync & Drift Detection

- Argo CD **automatically detects drift** between the live cluster state and Git state.
- It can either alert you or **automatically sync changes** without manual intervention.
- This ensures your cluster is *always* aligned with the declared configuration in Git, improving reliability and compliance.
- Many traditional CI/CD tools don't have this native drift detection and self-healing capability for Kubernetes resources.

3. Multi-Cluster and Multi-App Management from a Single Control Plane

- Argo CD supports managing **multiple Kubernetes clusters** from a single Argo CD instance.
- You get a centralized dashboard that shows all your apps and clusters, with status, history, and health info.
- This is much harder to achieve with traditional CI/CD tools, which usually target a single environment or cluster and require custom scripting.

4. Kubernetes Native & Lightweight

- Argo CD runs *inside* Kubernetes as a set of controllers and services.
- It's designed specifically for Kubernetes environments — no extra layers or abstractions.
- Traditional CI/CD systems are often standalone servers or cloud services that need integrations and plugins to talk to Kubernetes.

5. First-Class Support for Kubernetes Configuration Tools

- Argo CD works **natively with Helm, Kustomize, Jsonnet**, and plain YAML.
- This means it understands your Kubernetes manifests and can intelligently manage complex overlays and templating.
- Many older CI/CD tools treat Kubernetes manifests as just files to be copied or commands to be run, without native awareness of Kubernetes config formats.

6. Strong Auditability and Compliance

- Since your Kubernetes state is driven solely by Git commits, you get a full **audit trail** of who changed what and when.
- Rollbacks are trivial by simply reverting Git commits.
- This level of traceability is more implicit and harder to enforce in classic CI/CD pipelines.

7. Integration with Progressive Delivery Tools

- Argo CD integrates well with other CNCF tools like **Argo Rollouts** for canary deployments, blue-green, and automated rollbacks.
- This means you can do advanced deployment strategies in a Kubernetes-native way.
- While other CI/CD tools may support these concepts, Argo CD + Argo Rollouts provide a seamless Kubernetes-first experience.

Summary:

Traditional CI/CD Tools	Argo CD
General-purpose, supports many runtimes	Kubernetes-native GitOps continuous delivery
Imperative deployments (scripts, pipelines)	Declarative, Git-driven deployment with drift detection
Mostly single-cluster, CI focus	Multi-cluster, focused on CD & cluster state sync
Needs plugins to handle Kubernetes	Native support for Helm, Kustomize, Jsonnet
Build & test automation primary	Focuses on syncing desired cluster state with Git
Limited cluster state visibility	Real-time app health, sync status UI and alerts
Manual or scripted rollbacks	Easy rollback via Git commits

In essence, Argo CD complements your existing CI tools by handling the "continuous delivery" and cluster state management in a Kubernetes-native, GitOps way — bringing automation, reliability, and traceability to Kubernetes deployments that traditional CI/CD tools can struggle with.

Describe an example of workflow where ArgoCD is used

Here's a simple but typical **example workflow** that shows how Argo CD fits into a modern GitOps-based Kubernetes deployment pipeline, often combined with a CI tool like Jenkins or GitHub Actions.

Example Workflow: Deploying a Microservice Using Argo CD

1. Developer writes code and pushes to Git repo

- A developer commits code changes to a microservice repository (e.g., `my-app`).
- A separate Git repo (or a subfolder) holds Kubernetes manifests or Helm charts describing how the microservice should be deployed.

2. CI system builds and pushes Docker image

- A CI tool (like Jenkins, GitHub Actions, CircleCI) detects the code commit.
- It runs build, test, and security scans.
- It builds a Docker image for the new version of the microservice.
- The Docker image is pushed to a container registry (e.g., Docker Hub, ECR).

3. CI updates Kubernetes manifests in Git

- The CI pipeline updates the Kubernetes manifests or Helm chart values in the **GitOps repo** to reference the new Docker image tag.
- For example, it modifies the `image:` tag in the Deployment YAML to `my-app:1.2.3`.
- The CI pipeline commits and pushes this change to the **GitOps repo** (separate repo dedicated to deployment manifests).

4. Argo CD monitors the GitOps repo

- Argo CD continuously watches the GitOps repo branch.
- It detects the new commit that updates the image tag in the manifest.

5. Argo CD syncs the cluster state

- Argo CD compares the current live Kubernetes cluster state with the desired state defined in Git.
- It sees the image tag has changed and performs a rolling update of the Deployment to the new image version.
- Argo CD ensures all Kubernetes resources are created or updated as defined.

6. Argo CD provides visibility and health status

- The developer or operations team can view the Argo CD dashboard or CLI.
- It shows application sync status, health checks, and history of deployments.
- If the new deployment fails health checks, Argo CD can alert or automatically rollback to the previous working version (if configured).

7. Optional: Progressive Delivery

- If integrated with Argo Rollouts, Argo CD can perform canary or blue-green deployments to gradually shift traffic to the new version safely.

Visual Summary

```
Developer Code Repo  -->  CI Builds Image --> CI Updates GitOps Repo (manifests)
                                        ↓
                          Argo CD Watches GitOps Repo
                                        ↓
                        Argo CD Syncs Cluster to Desired State
                                        ↓
                    Cluster Running New Version of Microservice
                                        ↓
                        Health & Status Monitored in Argo CD UI
```

Why this workflow rocks:

- **Separation of concerns:** Developers focus on code; CI handles builds and tests; Argo CD handles cluster deployment.
- **Single source of truth:** GitOps repo fully defines cluster state — audited, version-controlled, and reviewable.
- **Automation:** Deployment is automatic once manifests are updated.
- **Visibility:** Real-time status and history of deployments.
- **Rollback:** Easy and safe rollback through Git history.
- **Kubernetes-native:** Everything runs within Kubernetes ecosystem naturally.

True or False? ArgoCD supports Kubernetes YAML files but not other manifests formats like Helm Charts and Kustomize

False.

Argo CD supports **not only plain Kubernetes YAML files** but also **other manifest formats like Helm charts, Kustomize, Jsonnet**, and more. It's designed to work natively with these popular Kubernetes configuration management tools.

What "GitOps Repository" means in regards to ArgoCD?

In the context of **Argo CD**, the **"GitOps Repository"** refers to the **Git repository that holds the declarative configuration files defining the desired state of your Kubernetes applications and infrastructure**.

What does the GitOps Repository contain?

- Kubernetes YAML manifests (Deployments, Services, ConfigMaps, etc.)
- Helm charts or Helm chart values files
- Kustomize overlays and base configurations
- Jsonnet or other supported config files
- Any other files that describe how your apps should be deployed on Kubernetes

Why is it called GitOps Repository?

- This repo acts as the **single source of truth** for your Kubernetes environment.
- Argo CD continuously monitors this repo to detect changes.
- When you commit and push updates (e.g., new image versions, config changes), Argo CD automatically **syncs** your Kubernetes cluster to match this desired state.
- This embodies the **GitOps** principle: your **Git repo is the source of truth for your infrastructure and app configuration**, enabling auditable, version-controlled, and declarative deployments.

Summary:

Term	Meaning in Argo CD context
GitOps Repository	Git repo containing Kubernetes manifests/configs that define the desired state of your apps and cluster
Purpose	Acts as source of truth, triggers Argo CD to sync changes to the cluster
Managed formats	YAML, Helm charts, Kustomize, Jsonnet, etc.

So, when people say “GitOps repo” with Argo CD, they mean the Git repository that Argo CD tracks and uses to **drive all Kubernetes deployments and configuration automatically**.

What are the advantages in using GitOps approach/repository?

Using a **GitOps approach** (and its GitOps repository) for managing Kubernetes deployments and infrastructure offers several key advantages:

1. Single Source of Truth

- All desired infrastructure and application configurations live in **one place — the Git repository**.
- This repository becomes the definitive reference for your cluster state.
- Any change must go through Git, making it clear and auditable what the current intended state is.

2. Version Control & History

- Git keeps a **full history of changes** with commit messages, authors, timestamps.
- You can track who changed what, when, and why.
- Rolling back to a previous working state is as simple as reverting a commit.

3. Auditability & Compliance

- Every deployment or configuration change is tied to a Git commit.
- This makes audits and compliance easier, with clear records of changes.
- You avoid "snowflake" clusters where manual changes create drift and confusion.

4. Automation & Reliability

- Tools like Argo CD automatically **sync the live cluster to match the Git repo state**.
- This eliminates manual intervention or ad-hoc scripts, reducing human errors.
- The system can **detect drift** and self-heal to the declared state.

5. Collaboration & Code Review

- Changes to infrastructure/app configs go through Git workflows (branches, PRs, reviews).
- Teams can **collaborate, review, discuss, and approve changes** before they hit production.
- This improves quality and security of deployments.

6. Declarative Infrastructure

- Describing the desired state in code (YAML, Helm, Kustomize) fits perfectly with Kubernetes' declarative nature.
- You don’t script *how* to deploy; you specify *what* the desired end state is.
- Makes deployments **idempotent and predictable**.

7. Consistent Environments

- Using GitOps repositories helps ensure **development, staging, and production environments** stay consistent.
- You can promote changes through branches or folders, reducing “works on my machine” problems.

8. Improved Security

- Since all changes go through Git, you can enforce **access controls, review policies, and signed commits**.
- Secrets can be managed using tools integrated with GitOps workflows (sealed-secrets, HashiCorp Vault).
- Reduces the risk of manual misconfiguration or unauthorized cluster changes.

9. Faster Recovery

- In case of failures, you can **quickly roll back** the cluster state by reverting Git commits.
- Your infrastructure state is reproducible and recoverable.

10. Better Visibility

- GitOps tools (like Argo CD) provide dashboards showing the **desired vs actual state, sync status, and health of apps**.
- This gives teams real-time insight into deployments and cluster health.

In short, GitOps makes your Kubernetes deployments more transparent, reliable, secure, and easy to manage by using Git as the central control plane.

Sorina, one of the engineers in your team, made manual changes to the cluster that override some of the configuration in a repo traced by ArgoCD. What will happen?

If Sorina makes manual changes directly to the Kubernetes cluster that **override** the configuration defined in the GitOps repository managed by Argo CD, here's what will happen:

1. Argo CD Detects Drift

- Argo CD continuously **monitors the live cluster state** and compares it against the **desired state defined in the Git repository**.
- When it detects that the live cluster has diverged (drifted) from the GitOps repo configuration — in this case, because of Sorina's manual changes — Argo CD flags this as **"OutOfSync"**.

2. Sync Status Changes to OutOfSync

- The affected application(s) or resources in the Argo CD dashboard will show an **OutOfSync** status.
- This means the cluster state no longer matches what's declared in Git.

3. Manual or Automatic Reconciliation

Depending on your Argo CD settings:

- **If automatic sync is enabled:**
  - Argo CD will automatically **revert the manual changes** by syncing the cluster back to the Git state.
  - This means the manual override will be undone and the cluster returns to the version declared in Git.
- **If automatic sync is disabled:**
  - Argo CD will **alert the team** but won't fix the drift until someone manually triggers a sync.
  - This allows operators to review changes before reconciliation.

4. Visibility and Audit

- The discrepancy is clearly visible in Argo CD's UI and CLI.
- This helps your team spot unauthorized or unintended manual changes quickly.

5. Best Practice Reminder

- Because manual changes get overwritten or cause drift alerts, **direct manual edits to the cluster are discouraged in GitOps workflows.**
- Instead, all changes should be made via **pull requests and commits to the GitOps repo** to keep the source of truth consistent.
- This keeps your deployments reliable, auditable, and reproducible.

In summary: Sorina's manual cluster changes will trigger Argo CD's drift detection, causing the app to go OutOfSync, and depending on settings, Argo CD will either automatically or manually restore the cluster to match the Git repo state — effectively undoing the manual changes.

Nate, one of the engineers in your organization, asked whether it's possible if ArgoCD didn't sync for changes done manually to the cluster. What would be your answer?

By **default, Argo CD is designed to detect and reconcile any drift** between the live cluster state and the desired state declared in the Git repository. That means:

- If someone makes manual changes directly to the cluster, **Argo CD will notice and mark the application as OutOfSync**.
- Depending on configuration, Argo CD will either **automatically revert those manual changes** during the next sync or **wait for a manual sync** to restore the desired state.

But, is it possible to configure Argo CD *not* to sync or override manual changes?

- **Yes, but with caveats.**

1. **Ignore Differences Feature**
    - Argo CD has a feature called **"resource.customizations.ignoreDifferences"** that lets you specify certain fields or resources to ignore during comparison.
    - If configured properly, Argo CD will **not consider some manual changes as differences** and won't attempt to sync those specific changes.
    - This is useful for things like fields managed by other controllers or autoscalers.
2. **Pause Syncing**
    - You can **pause automatic syncs** for an application, which means Argo CD will detect drift but **won't fix it until you resume sync manually**.
    - This allows manual changes to persist temporarily but carries the risk of drift accumulating.
3. **Manual Mode**
    - Running Argo CD in manual sync mode means it **only updates the cluster when a human triggers it**.
    - But again, drift will still be detected and reported.

Important considerations:

- Using these options to *ignore or avoid syncing manual changes* reduces the **guarantees of GitOps** — namely, that Git is the single source of truth and that cluster state matches Git.
- Allowing manual overrides can lead to **configuration drift, unexpected behaviors, and harder troubleshooting**.
- The best practice is to **make all changes via Git** and treat manual edits as exceptions that should be minimized.

So, my answer to Nate:

Yes, Argo CD can be configured to ignore certain manual changes or avoid automatic syncing, but it's generally not recommended because it breaks the GitOps principle of a single source of truth. Instead, manual changes should be avoided, or if necessary, carefully controlled using features like ignored differences or paused syncs.

How cluster disaster recovery becomes easier with ArgoCD?

Argo CD can significantly simplify **cluster disaster recovery** thanks to its GitOps model. Here's how:

How Argo CD Makes Cluster Disaster Recovery Easier

1. Git Repository as the Single Source of Truth

- Your entire Kubernetes app and infrastructure configuration lives in Git.
- In case of cluster failure or disaster, you have a **version-controlled backup of the desired cluster state**.
- No need to rely on manual documentation or ad-hoc backups of the cluster's live state.

2. Rebuild Cluster from Scratch

- After recovering or provisioning a new Kubernetes cluster, you just need to:
  - Install Argo CD on the new cluster.
  - Point Argo CD to the GitOps repository.
- Argo CD will **automatically sync all applications and resources** to the exact state declared in Git.

3. Idempotent and Declarative Deployments

- Because manifests describe the *desired state*, repeated deployments are safe and idempotent.
- No risk of partial or inconsistent configurations after recovery.

4. History and Rollbacks

- Git's commit history lets you:
  - Roll back to a known good configuration if recent changes caused issues.
  - Recover exactly the state before the disaster.

5. Automated and Auditable

- The recovery process is largely automated — less manual error during disaster recovery.
- Every step is traceable through Git commits and Argo CD's logs.

6. Separation of Concerns

- Cluster infrastructure (nodes, networking) can be restored independently.
- Application and configuration restore is fully handled by Argo CD syncing the Git repo.

In short:

Argo CD turns disaster recovery into a matter of "spin up a new cluster, install Argo CD, and let it pull and apply all your configs from Git." This dramatically reduces downtime, complexity, and human error during recovery.

Ella, an engineer in your team, claims ArgoCD benefit is that it's an extension Kubernetes, it's part of the cluster. Sarah, also an engineer in your team, claims it's not a real benefit as Jenkins can be also deployed in the cluster hence being part of it. What's your take?

Ella's point:

Argo CD is an extension of Kubernetes and runs inside the cluster itself, which is a big benefit.

- This is true and important because Argo CD is designed **specifically for Kubernetes**.
- It uses the Kubernetes API extensively and integrates **deeply with native Kubernetes concepts** like Custom Resource Definitions (CRDs), RBAC, namespaces, etc.
- Being "in-cluster" means Argo CD can monitor and manage resources **with native Kubernetes security, scalability, and lifecycle management**.
- It can easily watch the cluster state continuously without external networking complexities.
- This tight integration **enables powerful GitOps workflows that feel "native" to Kubernetes**.

Sarah's point:

Jenkins can also run inside the cluster, so being "in-cluster" is not unique or a big benefit.

- This is also true — Jenkins or many other tools **can be deployed inside Kubernetes clusters as pods**.
- But Jenkins is fundamentally a general CI/CD automation server — it's **not built specifically for Kubernetes management or GitOps**.
- Jenkins requires plugins, scripts, and custom configurations to interact with Kubernetes APIs.
- It does not have the same **declarative, continuous reconciliation model** built-in that Argo CD has for syncing cluster state from Git.
- Simply running inside the cluster doesn't make Jenkins Kubernetes-native or provide the same seamless integration.

My take:

- **Being "part of the cluster" is definitely a strong advantage for Argo CD because it is Kubernetes-native and designed for declarative GitOps workflows that tightly couple with Kubernetes APIs.**
- Jenkins being deployable inside Kubernetes is a convenience, but Jenkins **does not inherently have Kubernetes-native GitOps features**.
- The real difference lies in the **purpose-built nature** and **native integration** of Argo CD with Kubernetes, not just the fact that it runs in the cluster.
- So, Ella is right about the benefit of Argo CD's design and integration, and Sarah is right that "running in the cluster" alone isn't unique, but that's only part of the story.

In short: Ella is right that Argo CD's tight Kubernetes integration and in-cluster operation are major benefits. Sarah is right that simply running inside the cluster doesn't automatically provide Kubernetes-native features like GitOps.

How the main resource in ArgoCD called?

The main resource in Argo CD is called an **Application**.

An **Argo CD Application** represents a Kubernetes application you want to deploy and manage. It defines:

- The **Git repository** URL and the path within it where the manifests live
- The **target Kubernetes cluster** and namespace where the app should be deployed
- The **sync policy** and other deployment behaviors

Argo CD continuously monitors and syncs the Application resource to ensure the live cluster state matches what's declared in Git

Explain what is an "Application" in regards to ArgoCD

In Argo CD, an **Application** is the fundamental Kubernetes custom resource that defines and manages a deployed application or set of Kubernetes resources

What exactly is an Argo CD Application?

- It represents a **declarative description** of a Kubernetes application that you want Argo CD to deploy and manage.
- It ties together the **source** (usually a Git repository containing manifests, Helm charts, or Kustomize configs) and the **destination** (the target Kubernetes cluster and namespace).
- It defines how Argo CD should **synchronize** the desired state in Git with the actual state in the cluster.

Key parts of an Argo CD Application:

Aspect	Description
Source repo	Git repository URL and path containing manifests
Target cluster	Kubernetes cluster where the app will be deployed
Namespace	Kubernetes namespace to deploy the resources
Sync Policy	Defines whether sync is automatic or manual
Health checks	Status of deployed resources (Healthy, Degraded, etc.)

What does Argo CD Application do?

- Continuously **monitors** the live resources in the cluster.
- Detects **drift** between Git's desired state and cluster's actual state.
- **Syncs** the cluster state to match Git, either automatically or manually.
- Provides **visibility** into app status, health, and history through Argo CD's UI or CLI.

Why is Application important?

It's the core **unit of deployment and management** in Argo CD, enabling GitOps workflows where Git is the single source of truth and the cluster state is kept in sync automatically.

Here's a simple example of an Argo CD **Application** manifest that deploys a Kubernetes app from a Git repo:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-sample-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example-org/my-k8s-app.git
    targetRevision: main
    path: manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: my-app-namespace
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Explanation:

- `metadata.name`: The name of the Application resource (`my-sample-app`).
- `metadata.namespace`: Usually `argocd` — where Argo CD is installed.
- `spec.project`: The Argo CD project this app belongs to (default is fine for most).
- `spec.source.repoURL`: Git repo URL containing the Kubernetes manifests.
- `spec.source.targetRevision`: Branch or tag to track (`main` here).
- `spec.source.path`: Folder path in the repo with the manifests.
- `spec.destination.server`: Kubernetes API URL of the target cluster (the in-cluster API server here).
- `spec.destination.namespace`: Namespace where the app resources will be deployed.
- `spec.syncPolicy.automated`: Enables automatic syncing:
    - `prune: true` removes resources that are no longer defined in Git.
    - `selfHeal: true` automatically reverts manual changes in the cluster.

How ArgoCD makes access management in the cluster easier?

Argo CD simplifies **access management** in Kubernetes clusters in several ways:

How Argo CD Makes Access Management Easier

1. Centralized Control via RBAC

- Argo CD has its own **Role-Based Access Control (RBAC)** system that controls who can do what **within Argo CD** (e.g., who can sync apps, create applications, delete apps).
- You can define fine-grained permissions for users and groups, controlling access to specific applications or actions.
- This helps centralize access policies related to app deployment and management **outside of Kubernetes cluster RBAC**.

2. GitOps Enforces Declarative Access

- Because Argo CD manages cluster resources **based on Git repositories**, users **do not need direct access to the Kubernetes API** to make changes.
- Developers or operators can modify applications by pushing commits or opening pull requests in Git — **no direct kubectl access required**.
- This reduces the need to grant broad cluster access, improving security.

3. Service Accounts for Deployment

- Argo CD runs inside the cluster with its own **service account**, which is granted specific Kubernetes RBAC permissions.
- Access can be scoped strictly to only what Argo CD needs to deploy/manage apps, limiting risk.

4. Multi-Cluster and Multi-Project Support

- Argo CD supports managing multiple clusters and grouping applications into projects.
- You can assign access permissions per project and per cluster, ensuring **users only see and control what they're allowed**.
- This segmentation is simpler than juggling multiple Kubernetes cluster RBAC configs.

5. Integration with SSO and External Identity Providers

- Argo CD supports **single sign-on (SSO)** via OAuth2, OIDC, LDAP, SAML, etc.
- This allows you to integrate Argo CD user access with your existing identity management systems.
- User authentication and group memberships can map directly to Argo CD roles, simplifying user management.

In short:

Argo CD abstracts and centralizes Kubernetes access management by enabling teams to control deployment permissions within Argo CD itself, restricting direct cluster access, enforcing GitOps workflows, and integrating with enterprise identity providers — all making cluster access safer, simpler, and more auditable.

Describe the purpose of the following section in a an Application YAML file

```
source:
 repoURL: https://github.com/bregman-arie/devops-exercises
 targetRevision: HEAD
 path: main
```

Here's the purpose of that `source` section in an Argo CD Application YAML:

Explanation of the `source` section

```
source:
  repoURL: https://github.com/bregman-arie/devops-exercises
  targetRevision: HEAD
  path: main
```

- **`repoURL`**:
  This is the URL of the Git repository where Argo CD will look for the Kubernetes manifests (or Helm charts, Kustomize configs) to deploy.
  Here, it points to the GitHub repo `https://github.com/bregman-arie/devops-exercises`.
- **`targetRevision`**:
  Specifies which Git revision Argo CD should use to fetch manifests. This can be a branch name, tag, or commit SHA.
  `HEAD` means the latest commit on the default branch (usually `main` or `master`).
- **`path`**:
  The directory path **inside the Git repo** where Argo CD should find the Kubernetes manifests to apply.
  In this example, it is the `main` folder inside that repository.

In short:

This section tells Argo CD **where in Git** to find the app configuration files it needs to deploy — the specific repo, branch/commit, and folder path. Argo CD then uses this info to sync and apply those resources into the target Kubernetes cluster.

Describe the purpose of the following section in a an Application YAML file

```
destination:
  server: http://some.kubernetes.cluster.svc
  namespace: devopsExercises
```

Here's the purpose of the `destination` section in an Argo CD Application YAML:

Explanation of the `destination` section

```
destination:
  server: http://some.kubernetes.cluster.svc
  namespace: devopsExercises
```

- **`server`**:
  This specifies the **API server URL** of the target Kubernetes cluster where Argo CD should deploy the application resources.
  In this example, `http://some.kubernetes.cluster.svc` is the cluster's Kubernetes API endpoint.
- **`namespace`**:
  This is the **Kubernetes namespace** within the target cluster where the application's resources (pods, services, deployments, etc.) will be created or updated.
  Here, it is the `devopsExercises` namespace.

In short:

This section tells Argo CD **where** (which Kubernetes cluster and namespace) it should apply the manifests it pulls from Git. It directs Argo CD's sync operation to the exact location in your infrastructure.

What CRD would you use if you have multiple applications and you would like to group them together logically?

If you want to group multiple Argo CD Applications together logically, you would use an **AppProject** Custom Resource Definition (CRD).

What is an AppProject?

- **AppProject** is a built-in Argo CD CRD designed to **group and organize multiple Applications**.
- It acts like a namespace-level or logical boundary for Applications.
- You can define things like:
  - Which Git repositories and cluster destinations the grouped Applications can use
  - Role-based access control (RBAC) policies for users on those Applications
  - Resource quotas and sync policies scoped to the project
- It helps manage permissions and policies at a group level instead of per individual Application.

Summary

Resource	Purpose
Application	Represents a single deployable app
AppProject	Groups multiple Applications logically

Absolutely! Here's a simple example of an **AppProject** manifest you can use in Argo CD:

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: devops-team
  namespace: argocd
spec:
  description: Project for DevOps team applications
  sourceRepos:
    - https://github.com/your-org/devops-apps.git
    - https://github.com/your-org/shared-lib.git
  destinations:
    - namespace: devops
      server: https://kubernetes.default.svc
    - namespace: staging
      server: https://staging.k8s.cluster.svc
  clusterResourceWhitelist:
    - group: '*'
      kind: '*'
  namespaceResourceBlacklist:
    - group: ''
      kind: ConfigMap
  roles:
    - name: devops-admin
      description: Admin role for DevOps team
      policies:
        - p, role:devops-admin, applications, *, devops-team/*, allow
      groups:
        - devops-admin-group
```

Explanation:

- `metadata.name`: Name of the project (`devops-team`).
- `sourceRepos`: Allowed Git repositories for applications in this project.
- `destinations`: Allowed Kubernetes clusters and namespaces where apps in this project can be deployed.
- `clusterResourceWhitelist`: Which cluster-scoped resources are allowed.
- `namespaceResourceBlacklist`: Resources disallowed inside namespaces.

`roles`: Defines RBAC roles, policies, and linked user groups for managing access.

True or False? ArgoCD sync period is 3 hours

False.

By default, Argo CD's sync (reconciliation) period is **not 3 hours** — it usually happens much more frequently, typically every **3 minutes** (180 seconds). This means Argo CD checks the Git repo and cluster state approximately every 3 minutes to detect and fix drift.

You can customize the sync interval for each Argo CD Application by setting the **`syncOptions`** field in the Application spec.

For example, to change the refresh interval to 10 minutes, you can add:

```
spec:
  syncPolicy:
    syncOptions:
      - CreateNamespace=true
      - PruneLast=true
  # Custom refresh interval (in seconds)
  syncOptions:
    - RefreshInterval=600
```

However, more typically, to control the **automatic refresh interval** (how often Argo CD checks for changes in Git and cluster state), you configure this via the Argo CD ConfigMap or CLI settings globally, not usually per Application.

If you want a per-Application **manual sync interval**, you'd typically script or trigger syncs as needed, but the built-in polling interval is a global setting.

Describe shortly what ArgoCD does every sync period

Every sync period, **Argo CD** does the following:

1. **Fetches the desired state** from the configured Git repository (the manifests or Helm/Kustomize configs).
2. **Compares** this desired state with the current live state of the resources in the target Kubernetes cluster.
3. **Detects any differences (drift)** between Git and the cluster.
4. If configured to auto-sync, **applies changes** to the cluster to make the live state match the Git state.
5. Updates the **Application status and health** in Argo CD UI to reflect the current sync state.

In short: Argo CD continuously ensures the cluster matches the Git repository, automatically or on demand, during each sync period.

You deployed a new application in a namespace called "yay" but when running `kubectl get ns yay` you see there is no such namespace. What happened?

If you deployed an Argo CD Application targeting the namespace `"yay"` but when you run `kubectl get ns yay` it doesn't exist, here's what likely happened:

Possible reasons:

1. **Namespace Does Not Exist Yet**
   The namespace `"yay"` wasn't created beforehand, and your manifests do **not include the namespace resource**.
2. **Argo CD Did Not Create Namespace Automatically**
   By default, Argo CD **does not create namespaces** unless you enable the option `spec.syncPolicy.syncOptions` with `CreateNamespace=true` in the Application manifest.
   Without this, Argo CD can't deploy resources into a namespace that doesn't exist.
3. **Permissions Issue**
   Argo CD's service account might **not have permission** to create namespaces, so even if `CreateNamespace=true` is set, it failed silently.

How to fix:

- Add this under your Application spec to let Argo CD create the namespace automatically:

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
  syncOptions:
    - CreateNamespace=true
```

- Or manually create the namespace before deploying:

```
kubectl create namespace yay
```

- Check Argo CD logs or UI for errors related to permissions or sync failures.

Create a new application with the following properties:

app name: some-app
repo: https://fake.repo.address
app path: ./app_path
namespace: default
cluster: my.kubernetes.cluster

Here's a sample Argo CD Application manifest based on your properties:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: some-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://fake.repo.address
    targetRevision: HEAD
    path: app_path
  destination:
    server: https://my.kubernetes.cluster
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

- `metadata.name`: `some-app` — your app name
- `metadata.namespace`: usually `argocd` (where Argo CD is installed)
- `spec.source.repoURL`: your Git repo URL
- `spec.source.path`: folder path in repo (`./app_path` simplified as `app_path`)
- `spec.destination.server`: cluster API URL (`https://my.kubernetes.cluster`)
- `spec.destination.namespace`: Kubernetes namespace to deploy (`default`)
- `syncPolicy.automated`: auto sync enabled with pruning and self-healing

List all argocd apps

To list all Argo CD applications, use this CLI command:

```
argocd app list
```

This will display a table with all apps managed by Argo CD, showing their names, project, sync status, health, and more.

Print detailed information on the app called "some-app"

To print detailed information about the Argo CD application named **some-app**, run:

```
argocd app get some-app
```

This shows full details including sync status, health, source repo, destination cluster/namespace, history, and any errors.

How to add an additional (external) cluster for ArgoCD to manage?

To add an additional external Kubernetes cluster to Argo CD so it can manage applications on that cluster, you need to **register** the cluster with Argo CD.

Here's how to do it:

1. **Get kubeconfig credentials for the external cluster.**
   Make sure you have admin access to that cluster and the kubeconfig file or context.
2. **Run the `argocd cluster add` command:**

   ```
   argocd cluster add <CONTEXT-NAME>
   ```

   - `<CONTEXT-NAME>` is the name of the cluster context in your kubeconfig file for the external cluster.

   For example:

   ```
   argocd cluster add external-cluster-context
   ```

3. **What this does:**
   - Registers the external cluster with Argo CD's management.
   - Creates a service account and RBAC bindings in the external cluster.
   - Stores the cluster credentials in Argo CD for use by Applications.
4. **Verify:**
   After adding, you can list clusters known to Argo CD:

   ```
   argocd cluster list
   ```

Notes:

- You need `kubectl` configured with access to the external cluster.
- You must be logged in to Argo CD CLI (`argocd login`) before running the command.
- Once registered, you can use the external cluster's API server URL in Application manifests under `destination.server`.

How to list all the clusters ArgoCD manage?

To list all the Kubernetes clusters that Argo CD manages, use the CLI command:

```
argocd cluster list
```

This will display a table of all clusters registered with Argo CD, showing cluster names, server URLs, connection status, and whether they are the current context.

Here's what the typical output of `argocd cluster list` looks like and how to interpret it:

```
NAME                        SERVER                                                  STATUS
MESSAGE
https://kubernetes.default.svc   https://kubernetes.default.svc
Successful Connected to cluster
https://my.kubernetes.cluster    https://my.kubernetes.cluster
Successful Connected to cluster
```

Columns:

- **NAME**
  The cluster name or the API server URL. This is the identifier used in Argo CD.
- **SERVER**
  The Kubernetes API server endpoint URL of the cluster.
- **STATUS**
  Shows if Argo CD can successfully connect to and manage the cluster.
  - `Successful` means Argo CD has access and the cluster is reachable.
  - Other statuses like `Failed` or `Unknown` mean connection or permission issues.
- **MESSAGE**
  Additional details about the connection or errors if any.

Managing Clusters:

- **Add a new cluster:**
  Use `argocd cluster add <context>` as explained earlier.
- **Remove a cluster:**

  ```
  argocd cluster rm <cluster-name>
  ```

  This unregisters the cluster from Argo CD (does not delete the cluster itself).

- **Check cluster details:**
  Use:

  ```
  kubectl config get-contexts
  ```

  to see cluster contexts available locally.

- **Use specific cluster in Applications:**
  Specify the cluster's API server URL in the Application manifest's `destination.server`.

You've three clusters - dev, staging and prod. Whenever you update the application GitOps repo, all three clusters are being updated. What's the problem with that and how to deal with it?

Problem:

If **all three clusters (dev, staging, prod)** get updated simultaneously on every Git repo change, you risk:

- **Unintended production changes:** Bugs or untested code from dev can get deployed directly to prod.
- **No environment isolation:** You lose control over the release lifecycle and proper testing phases.
- **Hard to do staged rollouts:** You can't promote changes gradually from dev → staging → prod.

How to deal with it:

Use environment-specific Git branches or directories + separate Argo CD Applications per environment:

1. **Separate Git paths or branches per environment**
   - For example:
     - `repo/dev/` — manifests for dev cluster
     - `repo/staging/` — manifests for staging
     - `repo/prod/` — manifests for production
2. **Create distinct Argo CD Applications for each cluster/environment**
   - Each app points to its own path or branch in the repo.
   - Each app targets the appropriate cluster (`destination.server`) and namespace.
3. **Control promotion with Git workflows**
   - Developers push changes to `dev` branch or folder → dev cluster syncs automatically.
   - After testing, changes merge or copy to `staging` branch/folder → staging cluster syncs.
   - Once verified, changes promoted to `prod` branch/folder → production cluster syncs.

Optional enhancements:

- Use Argo CD's **sync waves or sync hooks** to orchestrate complex deployments per environment.
- Implement **approval gates or manual sync** for production environment to avoid accidental deployments.
- Use **App of Apps pattern** to manage multiple environments centrally.

This approach ensures **safe, controlled, and auditable environment promotion** aligned with best GitOps practices.

Git repo structure and **Argo CD Application manifests** for managing **dev, staging, and prod clusters separately**, following the GitOps best practices.

1. Git Repo Structure (e.g., `ecommerce-gitops`)

```
markdown
CopyEdit
ecommerce-gitops/
├── dev/
│   ├── frontend/
│   ├── backend/
│   └── common/
├── staging/
│   ├── frontend/
│   ├── backend/
│   └── common/
└── prod/
    ├── frontend/
    ├── backend/
    └── common/
```

- Each environment folder (`dev`, `staging`, `prod`) contains Kubernetes manifests tailored for that environment.
- `common/` can hold shared configmaps, secrets (encrypted), or base manifests.

2. Argo CD Application Manifests for Each Environment

Dev Application (manages dev cluster)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ecommerce-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/ecommerce-gitops
    targetRevision: HEAD
    path: dev
  destination:
    server: https://dev.k8s.cluster.api    # Dev cluster API server
    namespace: ecommerce-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Staging Application (manages staging cluster)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ecommerce-staging
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/ecommerce-gitops
```

```
    targetRevision: HEAD
    path: staging
  destination:
    server: https://staging.k8s.cluster.api   # Staging cluster API server
    namespace: ecommerce-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Prod Application (manages prod cluster)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: ecommerce-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-org/ecommerce-gitops
    targetRevision: HEAD
    path: prod
  destination:
    server: https://prod.k8s.cluster.api   # Production cluster API server
    namespace: ecommerce-prod
  syncPolicy:
    automated:   # You might want to disable automated sync here for manual approval
      prune: true
      selfHeal: true
```

3. Workflow Overview

1. Developers commit and push changes to `dev` folder → changes automatically sync to **dev cluster**.
2. After testing, changes are copied or merged into `staging` folder → changes sync to **staging cluster**.
3. When staging is validated, changes move to `prod` folder → sync happens on **prod cluster** (manual approval recommended).

Optional: Automate Promotion With GitHub Actions (high-level)

- **On dev merge:** run tests → on success, open PR to `staging`.
- **On staging merge:** run integration tests → on success, open PR to `prod`.
- **On prod merge:** trigger manual sync in Argo CD or wait for manual approval.

What are some possible health statuses for an ArgoCD application?

ArgoCD applications can have several health statuses that indicate the current state of the application in the cluster. The most common **health statuses** are:

1. **Healthy**
   - All resources are in their desired state and functioning correctly.
   - No errors or warnings detected.
2. **Degraded**
   - Some resources are not in the desired state or have errors.
   - Examples: Pods crashing, Deployment not fully rolled out, failed readiness/liveness probes.
3. **Progressing**
   - The application is in the middle of an update or deployment.
   - Resources are being created, updated, or deleted.
4. **Suspended**
   - The application has been suspended (e.g., manual pause of sync).
   - No ongoing sync actions.
5. **Unknown**
   - ArgoCD cannot determine the health status, often due to communication issues or missing resource information.

Explain manual syncs vs. automatic syncs

Manual Sync

- **Triggered explicitly by a user** via the ArgoCD UI, CLI (`argocd app sync`), or API.
- You review the changes first and decide when to apply them to the cluster.
- Useful when you want full control over deployment timing, for example:
  - In production environments with strict change control.
  - When you want to review changes before applying.
- ArgoCD compares Git repo state to cluster state and applies only the differences.
- If manual sync fails, you can troubleshoot and retry manually.

Automatic Sync

- ArgoCD is configured to **automatically apply changes** whenever it detects that the Git repository has updates (config or manifests).
- No user intervention needed; syncs happen at configured intervals or immediately upon detection.
- Useful for fast, continuous delivery workflows where you want changes deployed as soon as possible.
- Supports options like automatic pruning (removing resources deleted in Git) and self-healing (reverting manual cluster changes).
- Can be configured per application basis.

Summary Table

Aspect	Manual Sync	Automatic Sync
Trigger	User-initiated	Git change detected automatically
Control	High — manual approval needed	Low — hands-free deployment
Use Case	Production/stable environments	Continuous delivery/testing
Features	Requires manual retries	Can auto-prune and self-heal

Here's how you can **configure automatic sync** for an ArgoCD application by adding the `syncPolicy` section to your Application manifest.

Example: Enable Automatic Sync with Prune and Self-Heal

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: some-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://fake.repo.address
    targetRevision: HEAD
    path: ./app_path
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true      # Automatically delete resources that are removed from Git
      selfHeal: true   # Automatically revert manual changes on the cluster
```

Explanation:

- `automated`: Enables automatic syncing.
- `prune: true`: Deletes Kubernetes resources that are removed from Git, keeping the cluster clean.
- `selfHeal: true`: If someone manually changes resources in the cluster, ArgoCD will revert them to match Git.

How to Apply:

1. Save this manifest as `app.yaml`.
2. Apply it with:

```
kubectl apply -f app.yaml
```

Explain auto-pruning

Auto-pruning is a feature in ArgoCD that **automatically deletes Kubernetes resources from the cluster when they are removed from the Git repository**.

Why is Auto-Pruning Useful?

- When you update your Git repo and **remove manifests for certain resources**, without pruning those resources remain deployed in the cluster — causing drift.
- Auto-pruning ensures the cluster **stays in sync with the desired state declared in Git**, cleaning up any resources that no longer exist in the repo.
- This keeps your cluster **clean, predictable, and consistent** with the GitOps principle: *Git is the single source of truth*.

How Auto-Pruning Works

- During a sync (manual or automatic), ArgoCD compares the live cluster state with the manifests in Git.
- If a resource exists in the cluster but not in Git, ArgoCD will **delete** (prune) that resource automatically.
- Auto-pruning only works when enabled explicitly in the `syncPolicy` (e.g., `prune: true`).

Example snippet enabling auto-pruning in an Application manifest

```
syncPolicy:
  automated:
    prune: true
```

Important Notes

- Use auto-pruning carefully — it will delete resources not defined in Git, so ensure your Git repo has the full desired state.
- Can be combined with `selfHeal: true` for full automated reconciliation.

Explain self-heal in regards to ArgoCD

Self-heal is a feature in ArgoCD that **automatically detects and reverts manual or external changes made directly to Kubernetes resources in the cluster**, restoring them to the state defined in the Git repository.

Why is Self-Heal Useful?

- Sometimes engineers or processes make **manual changes directly to the cluster** (kubectl edits, interventions, or scripts).
- These changes cause **drift** between the actual cluster state and the desired state in Git.
- With **self-heal enabled**, ArgoCD continuously monitors resources and, if it detects any divergence, **automatically syncs back** to the Git state.
- This ensures the cluster state remains consistent with the GitOps single source of truth without requiring manual intervention.

How Self-Heal Works

- ArgoCD periodically compares live resources to their Git manifests.
- If differences are found due to manual changes or drift, ArgoCD triggers a sync **to revert those changes**.
- Self-heal only works when enabled in the Application's `syncPolicy`.

Example snippet enabling self-heal in an Application manifest

```
syncPolicy:
  automated:
    selfHeal: true
```

Combined with Auto-Pruning

- Usually, self-heal is combined with **auto-pruning** (`prune: true`) so that ArgoCD not only reverts manual changes but also removes unwanted resources no longer in Git.

Summary

Feature	What it Does	Benefit
Auto-Prune	Deletes resources removed from Git	Keeps cluster clean and aligned
Self-Heal	Reverts manual/external resource changes	Maintains cluster consistency automatically

To add **self-heal** to your ArgoCD Application configuration, you need to include it under the `syncPolicy.automated` section in your Application manifest.

Example: Adding Self-Heal to an ArgoCD Application

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: some-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://fake.repo.address
    targetRevision: HEAD
    path: ./app_path
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true       # Optional: auto-remove deleted resources
      selfHeal: true    # Enables self-healing of manual cluster changes
```

How to apply it?

1. Save the above YAML as `app.yaml`
2. Apply it:

```
kubectl apply -f app.yaml
```

What support is provided in ArgoCD for Helm?

ArgoCD provides **first-class support for Helm**, making it a powerful GitOps tool for managing Helm-based Kubernetes applications. Here's what ArgoCD offers for Helm:

Helm Support in ArgoCD

1. **Helm Chart as Source:**
   - You can specify a **Helm chart repo URL** or a **local Helm chart directory** in your Application manifest's `source`.
   - ArgoCD will fetch the Helm chart and render Kubernetes manifests from it.
2. **Helm Values Customization:**
   - You can pass **custom Helm values** (`values.yaml` overrides) directly in the Application manifest.
   - Supports both inline values and values files from the repo.
3. **Helm Parameters Overrides:**
   - You can override specific Helm chart parameters dynamically using ArgoCD's `helm.parameters` field.
4. **Support for Helm Repositories:**
   - ArgoCD can authenticate and pull charts from private Helm repositories.
5. **Helm Hooks & Lifecycle:**
   - Supports Helm lifecycle hooks as part of the sync process.
6. **Helm Chart Version Control:**
   - You can pin to specific chart versions (`chart.version`) or track `latest`.
7. **Helm CLI is Not Required:**
   - ArgoCD does not require Helm CLI installed on the cluster or the client; it uses a built-in Helm rendering engine.

Example snippet for Helm in Application manifest:

```
spec:
  source:
    repoURL: https://charts.bitnami.com/bitnami
    chart: nginx
    targetRevision: 9.3.7
    helm:
      values: |
        replicaCount: 3
        service:
          type: LoadBalancer
```

Summary

Feature	Description
Chart Source	Public or private Helm repo or local chart
Values Overrides	Inline values or values files
Parameter Overrides	Fine-tuned chart parameters
Version Control	Pin chart versions or track branches
No Helm CLI Dependency	Uses internal Helm rendering engine

Example ArgoCD Application Manifest for a Helm Chart

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-helm-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://charts.bitnami.com/bitnami  # Helm chart repo URL
    chart: nginx                                 # Helm chart name
    targetRevision: 9.3.7                        # Helm chart version
    helm:
      values: |                                  # Custom Helm values overrides
        replicaCount: 2
        service:
          type: LoadBalancer
  destination:
    server: https://kubernetes.default.svc       # Kubernetes API server URL
    namespace: default                            # Namespace to deploy into
  syncPolicy:
    automated:
      prune: true                                # Auto-delete resources removed from
Helm chart
      selfHeal: true                             # Auto-revert manual changes
```

Explanation

- **repoURL:** The Helm chart repository URL.
- **chart:** Name of the Helm chart to deploy.
- **targetRevision:** The specific version of the Helm chart.
- **helm.values:** YAML snippet overriding the chart's default values (like replica count and service type).
- **destination:** Cluster and namespace where the chart is deployed.
- **syncPolicy:** Automates syncing, pruning, and self-healing.

How to Apply

1. Save this manifest as `my-helm-app.yaml`.
2. Apply with:

```
kubectl apply -f my-helm-app.yaml
```

What is Argo Rollouts?

Argo Rollouts is a Kubernetes controller and set of CRDs that provide **advanced deployment strategies** for applications running on Kubernetes. It extends the native Kubernetes deployment capabilities with progressive delivery features like **blue-green**, **canary**, and **experiment** deployments.

Key Features of Argo Rollouts:

- **Canary Deployments:** Gradually shift traffic to a new version with automated analysis and rollback capabilities.
- **Blue-Green Deployments:** Run two separate environments (blue and green), switch traffic instantly when ready.
- **Traffic Routing:** Integrates with service meshes and ingress controllers (like Istio, NGINX, Linkerd) to control traffic flow during rollouts.
- **Automated Analysis:** Use metrics and custom analysis jobs to decide whether to promote or rollback a deployment.
- **Pause and Resume:** You can pause a rollout at any step to verify stability.
- **Integration with Argo CD:** Argo Rollouts can be used alongside Argo CD for GitOps-driven progressive delivery.

Why Use Argo Rollouts?

Native Kubernetes Deployments only support **basic rolling updates**. Argo Rollouts adds richer deployment patterns that reduce risk and downtime during updates by controlling how new versions get deployed and verified.

Here's a simple example of an **Argo Rollouts Canary deployment** manifest for an nginx application:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
  namespace: default
spec:
  replicas: 3
  strategy:
    canary:
      steps:
        - setWeight: 20       # Shift 20% traffic to new version
        - pause:
            duration: 1m      # Wait 1 minute for validation
        - setWeight: 50       # Shift 50% traffic to new version
        - pause:
            duration: 2m      # Wait 2 minutes
        - setWeight: 100      # Shift 100% traffic to new version
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21.1
```

```
        ports:
          - containerPort: 80
```

Explanation:

- **kind: Rollout** — This resource replaces Deployment for advanced rollout.
- **strategy.canary.steps** — Defines incremental traffic weight shifts and pauses.
- **setWeight:** directs percentage of traffic to the new version.
- **pause:** wait time for monitoring or manual approval.
- **replicas:** number of pods.
- **selector/template:** standard pod template.

What happens when you rollout a new version of your app with argo rollouts?

When you rollout a new version of your app using **Argo Rollouts**, here's what typically happens step-by-step:

1. New Version Detected

You update the container image tag (or other pod template fields) in the Rollout manifest and apply it.

2. Create New ReplicaSet

Argo Rollouts creates a new ReplicaSet for the new version of your app while keeping the old ReplicaSet(s) running.

3. Traffic Shift Begins (Canary or Blue-Green)

Based on your deployment strategy:

- **Canary:** Argo Rollouts shifts a portion of traffic to the new ReplicaSet incrementally using defined steps.
- **Blue-Green:** It creates the new version alongside the old one and waits for your manual or automatic switch.

4. Pauses & Analysis

If configured, Argo Rollouts pauses at each step for:

- Automated metric analysis (e.g., success rate, latency).
- Manual approval or validation.

5. Promotion or Rollback

- If analysis passes, Argo Rollouts continues shifting traffic until 100% is on the new version.
- If analysis fails or manual intervention triggers rollback, it shifts traffic back to the old version and scales down the failed ReplicaSet.

6. Cleanup

After a successful rollout, old ReplicaSets are scaled down and optionally cleaned up based on configuration.

Summary

Argo Rollouts adds **fine-grained control** over how new app versions get deployed and validated, reducing risk and downtime by progressively shifting traffic and enabling automated or manual promotion/rollback.

Native Kubernetes Deployment Rollout

- **Strategy:** Primarily supports **RollingUpdate** and **Recreate** strategies.
- **RollingUpdate:** Kubernetes replaces old pods with new pods incrementally, maintaining desired replicas.
- **Traffic Shift:** Happens automatically by replacing pods, but **no control over traffic percentages** — traffic routing is all-or-nothing (old pods fully replaced by new pods gradually).
- **No Built-in Progressive Delivery:** No native support for canary releases, blue-green deployments, or automated analysis.
- **No Traffic Routing Control:** Kubernetes itself doesn't manage traffic split; you'd need service mesh or ingress to do manual routing.
- **Rollbacks:** Possible, but usually manual and without automated metrics analysis.
- **No Built-in Pauses:** No steps or pauses to verify health mid-rollout.

Argo Rollouts

- **Advanced Strategies:** Supports **Canary**, **Blue-Green**, and **Experiment** strategies natively.
- **Traffic Shifting:** Fine-grained control to shift traffic by percentage between old and new versions.
- **Pausing and Validation:** Supports automatic pauses during rollout for manual approval or automated metric analysis.
- **Automated Promotion/Rollback:** Integrates with monitoring systems to automatically decide if rollout should proceed or rollback.
- **Traffic Routing Integration:** Works with service meshes and ingress controllers for real traffic routing control.
- **Self-Healing:** Can revert changes if new versions cause issues, without manual intervention.

Summary Table

Feature	Native Kubernetes Deployment	Argo Rollouts
Deployment Strategies	RollingUpdate, Recreate	Canary, Blue-Green, Experiment
Traffic Control	None (pod-level only)	Percentage-based traffic routing
Pausing / Steps	No	Yes
Automated Analysis	No	Yes
Auto Promotion/Rollback	No	Yes
Integration with Service Mesh	No	Yes

Bottom line: Argo Rollouts brings **progressive delivery features** missing in Kubernetes deployments, allowing safer, more controlled updates.

Scott, an engineer in your team, executes manually some smoke tests and monitors rollouts every time a new version is deployed. This way, if there is an issue he detects, he performs a rollback. What better approach you might suggest him to take?

I'd suggest Scott automate the monitoring and rollback process using **Argo Rollouts' built-in automated analysis and rollback features**. Here's why and how:

Better Approach for Scott:

1. **Set Up Automated Metrics Analysis:**
   Configure Argo Rollouts to integrate with monitoring tools (like Prometheus) to automatically evaluate health metrics (e.g., error rates, latency) during each rollout step.
2. **Define Rollout Steps with Pauses:**
   Use rollout steps with automatic or manual pauses to let the system verify stability before proceeding.
3. **Enable Automated Rollbacks:**
   If the metrics indicate degradation or failures, Argo Rollouts can automatically rollback to the previous stable version without manual intervention.
4. **Reduce Manual Work & Risk:**
   This automation frees Scott from constantly watching and manually testing, reduces human error, and accelerates safe deployments.

Summary:

Instead of manual smoke testing and monitoring, automate these checks with Argo Rollouts to get faster feedback and safer, self-healing deployments.

Here's a simple example of an **Argo Rollouts manifest** that includes **automated analysis and rollback** using Prometheus metrics:

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: nginx-rollout
  namespace: default
spec:
  replicas: 3
  strategy:
    canary:
      steps:
        - setWeight: 20
        - pause: {}
        - analysis:
            templates:
              - success-rate
            threshold: 1
            failureLimit: 1
        - setWeight: 50
        - pause: {}
        - analysis:
            templates:
              - success-rate
```

```
            threshold: 1
            failureLimit: 1
        - setWeight: 100
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21.1
          ports:
            - containerPort: 80
---
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
  namespace: default
spec:
  metrics:
    - name: request-success-rate
      interval: 30s
      successCondition: result > 0.95
      failureCondition: result < 0.95
      provider:
        prometheus:
          address: http://prometheus-server.default.svc.cluster.local
          query: |
            sum(rate(http_requests_total{job="nginx",status=~"2.."}[1m])) /
            sum(rate(http_requests_total{job="nginx"}[1m]))
```

Explanation:

- **Rollout with Canary strategy:** Traffic shifts 20%, then 50%, then 100%.
- **Pause steps:** Wait for analysis after traffic shift.
- **Analysis step:** Runs Prometheus query to check if success rate of requests is >95%.
- **FailureLimit:** If analysis fails once, rollback triggers automatically.
- **AnalysisTemplate:** Defines the metric and query to evaluate the health of the rollout.

This setup automates monitoring and rollback, reducing manual checks and improving reliability.

Explain the concept of "Analysis" in regards to Argo Rollouts

In **Argo Rollouts**, **"Analysis"** is a powerful feature that lets you **automatically evaluate the health and performance of a new application version during a rollout** by running custom metric checks against monitoring systems like Prometheus, Wavefront, Datadog, etc.

What is Analysis?

- It's a **predefined set of metrics and queries** that run periodically during rollout steps.
- Each metric defines **success and failure conditions** based on the query results.
- Analysis runs **automatically at specified intervals or after traffic shifts** to assess whether the new version is behaving correctly.

Why use Analysis?

- To **detect regressions or issues early** during progressive delivery.
- To **automate promotion or rollback** decisions without manual intervention.
- To **ensure confidence** that the new version meets SLA or quality targets before full rollout.

How Analysis works in a Rollout:

1. You define an **AnalysisTemplate** specifying metrics and success/failure criteria.
2. Reference that template in the Rollout manifest at desired steps.
3. When the rollout reaches those steps, Argo Rollouts runs the analysis queries repeatedly.
4. If success criteria are met, rollout continues; if failure criteria are hit, rollback can trigger automatically.

Summary:

Analysis in Argo Rollouts turns your deployment into a **self-verifying, automated process** that monitors real-time application health, helping to deliver safer and more reliable updates.

Explain the following configuration

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  args:
  - name: service-name
  metrics:
  - name: success-rate
    interval: 4m
    count: 3
    successCondition: result[0] >= 0.90
    provider:
      prometheus:
        address: http:/some-prometheus-instance:80
        query: sum(response_status{app="{{args.service-
name}}",role="canary",status=~"2.*"})/sum(response_status{app="{{args.service-
name}}",role="canary"})
```

What it does:

- **apiVersion & kind**:
  Defines this as an `AnalysisTemplate` resource used by Argo Rollouts.
- **metadata.name = success-rate**:
  The name of this AnalysisTemplate, which can be referenced by Rollouts.
- **spec.args**:
  Defines a parameter called `service-name`. This lets you reuse the template for different services by passing the app name dynamically.
- **metrics**:
  Contains one metric named `success-rate` that will be evaluated during rollout analysis.

Metric details:

- **interval: 4m**
  The metric query runs every 4 minutes.
- **count: 3**
  The metric must be checked 3 times (i.e., three 4-minute intervals, totaling 12 minutes).
- **successCondition: result[0] >= 0.90**
  The metric is considered successful if the first query result (the value returned by the Prometheus query) is **at least 90%** (0.90).
- **provider.prometheus.address**:
  The Prometheus server URL where the query will be sent.
- **query**:
  A Prometheus expression to calculate the success rate:

  ```
  sum(response_status{app="{{args.service-name}}",role="canary",status=~"2.*"}) /
  sum(response_status{app="{{args.service-name}}",role="canary"})
  ```

  This means:

  - Numerator: Count of successful HTTP responses (`status` starting with 2xx) for the canary version of the app.

    - Denominator: Total HTTP responses for the canary version.
      The division gives the fraction of successful requests, i.e., the success rate.
- **Template Parameter** `{{args.service-name}}`
  This placeholder is replaced by the actual service name argument passed to the AnalysisTemplate when referenced.

Summary:

This `AnalysisTemplate` measures the **HTTP success rate** (2xx responses) of a specified canary service over a 12-minute period (3 intervals × 4 minutes). The rollout will consider this analysis successful if the success rate is at least 90%. Otherwise, the analysis will fail, potentially triggering rollback.

How to list rollouts?

To list all Argo Rollouts in your Kubernetes cluster, you can use the `kubectl` command with the Argo Rollouts custom resource:

```
kubectl get rollouts
```

If you want to list rollouts in a specific namespace (e.g., `default`), add the `-n` flag:

```
kubectl get rollouts -n default
```

This will show you the names, statuses, and other summary info of all Rollout resources.

If you have the `kubectl-argo-rollouts` plugin installed, you can also run:

```
kubectl argo rollouts list
```

Which provides enhanced commands for managing rollouts.

How to list the rollouts of a given application?

In Argo Rollouts, **Rollouts** are separate Kubernetes resources—you don't really have "rollouts of an application" as a list because each Rollout is itself an application workload.

So, to list the rollout for a **specific Rollout resource**, you just check that rollout by name:

```
kubectl get rollout <rollout-name> -n <namespace>
```

If by "rollouts of a given application" you mean listing rollouts related to an app by label or annotation, you can filter by labels, for example:

```
kubectl get rollouts -n <namespace> -l app=<app-name>
```

This lists all Rollouts in that namespace having the label `app=<app-name>`.

If your app only has one Rollout resource, it's a one-to-one relationship, so you'd just get that Rollout by name.

How to check the status of a rollout?

To check the status of a rollout, you can use:

```
kubectl argo rollouts get rollout <rollout-name> -n <namespace>
```

This command shows detailed rollout status including phases, steps, replicas, and conditions.

If you don't have the Argo Rollouts CLI plugin, you can use standard kubectl:

```
kubectl get rollout <rollout-name> -n <namespace> -o yaml
```

or

```
kubectl describe rollout <rollout-name> -n <namespace>
```

But the `kubectl argo rollouts get rollout` command is tailored for rollouts and gives a clearer, user-friendly view.

How do you monitor a rollout?

Monitoring an Argo Rollouts rollout involves tracking the progress, health, and status of the rollout to ensure the new version is deployed successfully and safely. Here's how you typically monitor it:

1. Using `kubectl argo rollouts` CLI

- Check rollout status and progress:

```
kubectl argo rollouts get rollout <rollout-name> -n <namespace>
```

This shows details like:

- Current step in the rollout
- Number of pods updated
- Traffic split (for Canary)
- Health status of the rollout
- Watch rollout events live:

```
kubectl argo rollouts watch <rollout-name> -n <namespace>
```

This streams real-time updates until rollout completes or fails.

2. Argo Rollouts Dashboard (Web UI)

- Argo Rollouts has a dedicated UI plugin (a web dashboard) which provides a visual interface to:
  - See rollout status, history, and details
  - Promote or abort rollouts manually
  - Inspect analysis runs and metrics

You can access it via:

```
kubectl argo rollouts dashboard
```

3. Using Metrics and Analysis

- Argo Rollouts supports **analysis templates** that query metrics from systems like Prometheus.
- You can configure the rollout to automatically analyze health metrics (e.g., error rate, latency).
- Rollouts can automatically pause or rollback based on metric results.

4. Alerts and Notifications

- Integrate with monitoring tools (Prometheus Alertmanager, Slack, PagerDuty) to notify the team about rollout progress or failures.
- Use tools like **Argo Events** to trigger notifications on rollout state changes.